

AXIOM

Language Specification

Version 1.3

CONCEPT Construct . Language Validator

Release date:	April 15, 2026
Project:	AXIOM -- An AI-Native Language for Self-Evolving Intelligence
Author:	Antonio Roberts
Previous version:	v1.2 -- evaluator.axiom & rewriter.axiom upgrades

Abstract

Version 1.3 introduces two foundational capabilities to the Axiom language. First, the **CONCEPT** construct enables language-native extensibility by defining structured, named ideas with **PURPOSE**, **APPLIES WHEN**, **REQUIRES**, and **EFFECT** fields. Concepts are automatically activated per-task based on keyword overlap, injecting relevant constraints into the agent's system prompt without modifying its base definition. Second, the Language Validator enforces structural, purity, and semantic constraints on any **.axiom** file through three validation phases, producing a **STATUS** (valid / warning / invalid) with actionable **ISSUES** and **SUGGESTIONS**. Together these features establish Axiom as a self-describing, self-enforcing AI-native language.

1 Overview

Version 1.3 extends the Axiom language along two independent but complementary axes. The CONCEPT construct provides a structured mechanism for adding new native ideas to the language without modifying agent definitions or overloading existing constraints. The Language Validator enforces the purity and structure of all .axiom files, catching drift toward external programming paradigms and flagging vague or procedural language before it reaches the model.

1. Introduces the CONCEPT construct for language-native extensibility.
2. Adds a three-phase validation layer (syntax, purity, semantic).
3. Separates language growth (CONCEPT) from constraint enforcement (Validator).
4. Provides a shared concept library (concepts.axiom) seeded with UncertaintyBound and RewardGuard.
5. Wires validation output into both the CLI (run_axiom.py) and the Streamlit UI (ui.py).

Version History

v1.3 (current)	CONCEPT construct, Language Validator, concepts.axiom seed library
v1.2	evaluator.axiom + rewriter.axiom upgrades -- GOAL, PROCESS, FAILURE, SUCCESS, shar
v1.1	worker.axiom upgrades -- FAILURE block, OUTPUT block, sharper CONSTRAINTS
v1.0	DSL v0 -- parser.py, worker/evaluator/rewriter.axiom, overlay system

2 CONCEPT Construct

The CONCEPT block is a new top-level construct in the Axiom language. It defines a named, structured idea that can be injected into an agent's system prompt when the task context matches. Concepts live in any .axiom file or in the shared concepts.axiom library and are activated automatically -- agents never need to be modified to benefit from new concepts.

2.1 Syntax

```
CONCEPT <Name>
PURPOSE      <one-line description of what this concept expresses>
APPLIES WHEN <space-separated keywords triggering this concept>
REQUIRES     <what the agent must include when this concept is active>
EFFECT       <how the agent's behaviour changes>
```

2.2 Fields

PURPOSE	Declares the intent of this concept in one line.
APPLIES WHEN	Space-separated keywords. If any appear in the task, the concept activates.
REQUIRES	The specific output requirement the agent must satisfy.
EFFECT	The behavioural constraint applied to the agent's response.

2.3 Seed Concepts (concepts.axiom)

CONCEPT UncertaintyBound	
PURPOSE	Express uncertainty in measurable form
APPLIES WHEN	

CONCEPT RewardGuard	
PURPOSE	Prevent reward hacking or proxy gaming
APPLIES WHEN	

2.4 Runtime Behaviour

1. get_prompt_with_concepts(agent_name, task) loads the agent .axiom and the shared concepts.axiom library.
2. detect_concepts(task, parsed) tokenises each concept's APPLIES WHEN field and checks for

overlap with the task string (min token length 4 to skip stop words).

3. Only matching concepts are retained; non-matching concepts are removed before rendering.
4. Active concepts are appended to the system prompt under an 'Active Concepts' section.
5. Concepts that do not match the task are completely invisible to the model -- zero noise.

2.5 Example -- Active Concept in System Prompt

Task: "Design a reward function that maximizes user engagement"

Active Concepts:

- CONCEPT RewardGuard: Rejects harmful optimization strategies -- agent must verify the optimized metric cannot be gamed without achieving the real goal
(applies when: reward optimization metric objective incentive maximize minimize utility proxy)

Task: "Write a summary of this article" -> no active concepts (prompt unchanged)

3 Language Validator

The Language Validator (`axiom_files/validator.py`) inspects any parsed `.axiom` dict and returns a structured result indicating whether the definition is safe to use. Validation is always non-blocking -- it produces warnings and errors for the operator to review, but never halts execution.

3.1 Output Schema

```
{
  "status":      "valid" | "warning" | "invalid",
  "issues":      [
    {
      "phase":    "syntax" | "purity" | "semantic",
      "level":    "error" | "warning",
      "field":    "<field name>",
      "message":  "<description>"
    }
  ],
  "suggestions": ["<actionable correction>", ...]
}
```

3.2 Phase 1 -- Syntax Validation

- AGENT field present and non-empty.
- At least one of PURPOSE or GOAL must be defined.
- VERSION matches the pattern N.N (e.g. 1.3).
- SUCCESS weights sum to 1.0 +/- 0.01 tolerance.
- MUTATES and CANNOT_MUTATE reference only known Axiom fields.
- Every CONCEPT block has all four sub-fields (PURPOSE, APPLIES WHEN, REQUIRES, EFFECT) populated.

3.3 Phase 2 -- Purity Validation

Scans every string value in the parsed dict for patterns that indicate drift into external programming paradigms. Any match is an error.

def <name>(	Python function definition
class <name>:	Python class definition
for <x> in	Procedural for-loop
while ...:	Procedural while-loop
import <module>	Import statement
return	return keyword
print(...)	print() call
:=	Walrus operator

lambda	Lambda expression
---------------	-------------------

3.4 Phase 3 -- Semantic Validation

- Vague qualifier detection in CONSTRAINTS and RULES: flags entries containing 'try to', 'consider', 'if possible', 'when needed', 'appropriate', 'reasonable', 'as needed', 'maybe', 'perhaps', 'generally', 'typically', 'usually' -- unless the entry also contains a numeric threshold (e.g. ' ≥ 7 points'). Level: warning.
- Procedural drift in PROCESS: entries containing 'if', 'else', 'while', 'loop', or 'return' violate the declarative requirement. Level: error.
- Constraint/rule overlap: exact duplicate text appearing in both CONSTRAINT and RULES sections. Level: warning.

3.5 Status Determination

The overall STATUS is set to 'invalid' if any issue has level 'error'. If all issues are 'warning', STATUS is 'warning'. If there are no issues, STATUS is 'valid'.

OK VALID

! WARNING (1 issue)

X INVALID (3 issues)

4 Integration

4.1 CLI -- run_axiom.py

At startup, after loading each agent's system prompt preview, run_axiom.py calls validate_file() and prints a coloured STATUS badge. Issues are listed inline. Execution continues regardless of status.

```
WORKER.axiom -> system prompt preview: ...
  Validator: ! WARNING (1 issue)
    [warn] [semantic] rules: Vague qualifier 'appropriate' ...

EVALUATOR.axiom -> system prompt preview: ...
  Validator: OK VALID

REWRITER.axiom -> system prompt preview: ...
  Validator: OK VALID
```

4.2 Streamlit UI -- DSL Tab

In the 'Current .axiom definitions' expander, each agent file now shows a colour-coded badge (? VALID / ? WARNING / ? INVALID) next to its name. When issues exist, a collapsible 'N validator issue(s)' expander shows the full issue list and suggestions.

4.3 Public API

validate(parsed)	Validate a pre-loaded parsed dict. Returns status/issues/suggestions.
validate_file(agent_name)	Load a .axiom file by agent name and validate it.
detect_concepts(task, parsed)	Return concept names whose APPLIES WHEN matches the task string.
get_prompt_with_concepts(a, task)	Load agent + concept library, filter by task, return system prompt.
get_prompt(agent_name)	Existing: load .axiom and return full system prompt.
get_prompt_with_overlays(a, lst)	Existing: load base + overlay files, merge, return system prompt.

5 Validation Rules Reference

The following table lists all validation rules in v1.3.

Rule	Phase	Level	Trigger
AGENT present	syntax	error	AGENT field missing or empty.
PURPOSE or GOAL	syntax	error	Neither PURPOSE nor GOAL defined.
VERSION format	syntax	warning	VERSION does not match N.N pattern.
SUCCESS sums to 1.0	syntax	warning	SUCCESS weights sum != 1.0 (+/-0.01).
Valid MUTATES fields	syntax	warning	MUTATES/CANNOT_MUTATE refs unknown field.
CONCEPT completeness	syntax	error	CONCEPT missing one or more of 4 sub-fields.
No def/class/for/while	purity	error	External code pattern found in any field.
No import/return/print	purity	error	Import, return, or print() found in any field.
No := or lambda	purity	error	Walrus operator or lambda found.
No vague qualifiers	semantic	warning	Vague term without numeric threshold in CONSTRAINTS/RULES.
Declarative PROCESS	semantic	error	Conditional or loop construct in PROCESS.
No CONSTRAINT/RULE overlap	semantic	warning	Exact duplicate in both CONSTRAINT and RULES.

6 File Reference

axiom_files/parser.py	Extended: CONCEPT parse/save/render, detect_concepts(), get_prompt_with_concepts()
axiom_files/validator.py	New: Language Validator -- validate(), validate_file()
axiom_files/concepts.axiom	New: Shared concept library -- UncertaintyBound, RewardGuard
run_axiom.py	Updated: validator badges printed at startup per agent
ui.py	Updated: validator badges + issue expander in DSL tab
CHECKLIST.txt	Updated: all v1.3 items marked done
axiom_files/worker.axiom	v1.2 -- upgraded FAILURE, OUTPUT, sharper CONSTRAINTS
axiom_files/evaluator.axiom	v1.2 -- GOAL, RULES, PROCESS, FAILURE, SUCCESS
axiom_files/rewriter.axiom	v1.2 -- GOAL, PROCESS, CHECK, FAILURE, OUTPUT, SUCCESS

7 Testing

7.1 Parser / CONCEPT

1. `python axiom_files/parser.py --` prints all 3 agent prompts. Confirms no errors from CONCEPT additions.
2. `python -c "from axiom_files.parser import load_axiom; p=load_axiom('concepts'); print([c['name'] for c in p['concepts']])"` -- confirms ['UncertaintyBound', 'RewardGuard'].
3. Call `get_prompt_with_concepts('worker', 'reward function')` -- confirm RewardGuard appears in Active Concepts.
4. Call `get_prompt_with_concepts('worker', 'Write a summary')` -- confirm no Active Concepts section.

7.2 Validator

1. `python axiom_files/validator.py --` prints STATUS for worker (! WARNING), evaluator (? VALID), rewriter (? VALID).
2. Inject 'def foo():' into any constraint, re-run -- confirm ? INVALID with purity error.
3. Inject 'try to be helpful' into a rule, re-run -- confirm ! WARNING with semantic warning.
4. Inject 'if task is unclear: ask' into PROCESS, re-run -- confirm ? INVALID with procedural-drift error.
5. Restore file after each test.

7.3 Integration

1. `python run_axiom.py --` coloured validator badges appear before the loop prompt.
2. `streamlit run ui.py --` DSL tab shows ? WARNING for worker.axiom, ? VALID for evaluator and rewriter.

8 Conclusion

Axiom v1.3 delivers two pillars of a self-describing AI language. The CONCEPT construct gives the language a native mechanism for growth -- new ideas can be added to the concept library without touching any agent definition, and they activate automatically when relevant. The Language Validator gives the language a native mechanism for self-enforcement -- structural errors, purity violations, and vague language are caught before they reach the model, maintaining the integrity of the DSL as it evolves. Together they form the foundation described in the v1.3 specification: a language that can grow and a language that can police itself.

Next planned work: .axiom version history (git-style diff log), export best run as standalone .axiom bundle, evaluator mutation loop, snapshot/restore at convergence.